



Zoro Challenge

Montar el entorno del desafío

Primero, debemos iniciar el entorno para completar el desafío para ello es necesario levantar el contenedor de docker utilizando el primer comando documentado en  [Utils](#).

Luego, debemos cambiar de usuario al correspondiente al reto, para ello utilizamos el segundo comando documentado en  [Utils](#).

```
PS C:\Users\Usuario\Documents\GitHub\data-encryption\ctf_onepiece_symmetric_cipher> docker exec -it zoro_challenge bash
nobody@1241ab859171:/home/zoro$ su zoro
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

zoro@1241ab859171:~$
```

La contraseña es la bandera del  [Luffy_Challenge](#).

Extraer bandera de **flag.txt**

El primer paso para extraer la bandera del desafío, es encontrar todos los posibles archivos que puedan contenerla, para ello, utilizamos el primer comando documentado en  [Utils](#).

```
zoro@1241ab859171:~/ONEPIECE$ find . -type f -iname "*flag*"
./East_Blue/Loguetown/Casa_de_Dragon/flag.txt
./East_Blue/Shell_Town/Casa_de_Morgan/flag.txt
./East_Blue/Romance_Dawn/Casa_de_Luffy/flag.txt
./Alabasta/Rainbase/Casa_de_Igaram/flag.txt
./Alabasta/Rainbase/Casa_de_Vivi/flag.txt
./Alabasta/Alubarna/Casa_de_Igaram/flag.txt
./Whole_Cake_Island/Caramel_Mountain/Casa_de_Pudding/flag.txt
./Whole_Cake_Island/Cacao_Island/Casa_de_Katakuri/flag.txt
./Whole_Cake_Island/Whole_Cake_Chateau/Casa_de_Big_Mom/flag.txt
./Pirate_Island/Pirates_Hideout/Casa_de_Shanks/flag.txt
./Pirate_Island/Pirates_Hideout/Casa_de_Gold_Roger/flag.txt
zoro@1241ab859171:~/ONEPIECE$
```

Luego de obtener todos los potenciales archivos, verificamos el contenido de cada uno, utilizando el comando `cat`, hasta encontrar el archivo que contiene texto cifrado.

```
zoro@1241ab859171:~/ONEPIECE$ cat ./East_Blue/Romance_Dawn/Casa_de_Luffy/flag.txt
2b882f5fa42928283bab513317242bb3a649cd9cf28abdc23ca64bb2ccedf7449c81e98141zoro@1241ab859171:~/ONEPIECE$
zoro@1241ab859171:~/ONEPIECE$
```

Para descifrar la bandera, utilizamos un script en Python que implementa el algoritmo RC4 para descifrar el texto cifrado utilizando mi carnet de estudiante (21542) como clave.

```
#!/usr/bin/env python3

import sys
import binascii

def KSA(key):
    """Algoritmo de programación de clave (Key Scheduling Algorithm)"""
    keylength = len(key)
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % keylength]) % 256
        S[i], S[j] = S[j], S[i]
```

```

return S

def PRGA(S, data_length):
    """Algoritmo pseudo-aleatorio de generación (Pseudo-Random Generation)
    i = 0
    j = 0
    keystream = []
    for _ in range(data_length):
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        K = S[(S[i] + S[j]) % 256]
        keystream.append(K)
    return keystream

def RC4(key, data):
    """Implementación del algoritmo RC4"""
    key = [ord(c) for c in key]
    S = KSA(key)
    keystream = PRGA(S, len(data))
    return bytes([data[i] ^ keystream[i] for i in range(len(data))])

def main():
    """Función principal del script"""
    if len(sys.argv) != 3:
        print(f"Uso: {sys.argv[0]} <ruta_del_archivo> <clave_rc4>")
        sys.exit(1)

    file_path = sys.argv[1]
    rc4_key = sys.argv[2]

    try
        with open(file_path, 'r') as file:
            hex_content = file.read().strip()

    try:
        binary_data = binascii.unhexlify(hex_content)
    except binascii.Error:

```

```

        print(f"Error: El contenido del archivo no es un hexadecimal válido")
        sys.exit(1)

    decrypted_data = RC4(rc4_key, binary_data)

    try:
        print(decrypted_data.decode('utf-8'))
    except UnicodeDecodeError:
        print(''.join(chr(b) if 32 <= b < 127 else '.' for b in decrypted_data))

except FileNotFoundError:
    print(f"Error: El archivo {file_path} no existe")
    sys.exit(1)
except Exception as e:
    print(f"Error: {str(e)}")
    sys.exit(1)

if __name__ == "__main__":
    main()

```

Creamos el script utilizando el sexto comando documentado en  [Utils](#) para luego otorgarle permisos de ejecución utilizando el séptimo comando documentado.

Finalmente, ejecutamos el script utilizando el octavo comando documentado en  [Utils](#).

```

zoro@1241ab859171:~/ONEPIECE$ nano decrypt.sh
zoro@1241ab859171:~/ONEPIECE$ chmod +x decrypt.sh
zoro@1241ab859171:~/ONEPIECE$ ./decrypt.sh ./East_Blue/Romance_Dawn/Casa_de_Luffy/flag.txt "21542"
FLAG_df8a0e00011738f8fd2863162cf72cd5
zoro@1241ab859171:~/ONEPIECE$ █

```

Obteniendo así, la bandera descriptada.

```
FLAG_df8a0e00011738f8fd2863162cf72cd5
```

Extraer párrafo de poneglyph.jpeg

Al igual que con la extracción de la bandera, el primer paso es encontrar la carpeta que contiene la imagen cifrada, para ello, utilizamos el cuarto comando documentado en  [Utils](#).

```
zoro@1241ab859171:~/ONEPIECE$ find . -type f -name "*poneglyph*"
./Pirate_Island/Pirates_Tavern/Casa_de_Shanks/poneglyph.zip
zoro@1241ab859171:~/ONEPIECE$ cd ./Pirate_Island/Pirates_Tavern/Casa_de_Shanks
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks$ ls
poneglyph.zip
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks$
```

Luego de obtener la carpeta, nos dirigimos hacia su directorio y la extraemos su contenido utilizando el quinto comando documentado en  [Utils](#). La contraseña a utilizar es la misma que la del usuario.

```
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks$ 7z x poneglyph.zip -oponeglyph

7-Zip [64] 16.02 : Copyright (c) 1999-2016 Igor Pavlov : 2016-05-21
p7zip Version 16.02 (locale=C,Utf16=off,HugeFiles=on,64 bits,8 CPUs Intel(R) Core(TM) i7-8650U CPU @ 1.90GHz (806EA),ASM,AES-NI)

Scanning the drive for archives:
1 file, 54098 bytes (53 KiB)

Extracting archive: poneglyph.zip
--
Path = poneglyph.zip
Type = zip
Physical Size = 54098

Enter password (will not be echoed):
Everything is Ok

Size:          53728
Compressed:    54098
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks$ ls
poneglyph poneglyph.zip
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks$
```

Para poder utilizar el comando de extracción, es posible que sea necesario instalar ciertas librerías. Para ello, utilice el noveno comando documentado en  [Utils](#).

Para descifrar el parrafo, utilizamos un script que:

1. Extrae el texto de la imagen.
2. Toma el texto de la imagen y realiza una operación XOR entre el texto cifrado y mi carnet de estudiante (21542).

```
#!/bin/bash

#Verificar que se proporcione el carné como argumento
if [ $# -ne 1 ]; then
    echo "Uso: $0 <tu_carné>"
    exit 1
fi

STUDENT_ID=$1

#Definir los nombres de los directorios y archivos
IMAGE_PATH="poneglyph.jpeg" # Asumiendo que está en el mismo directorio
OUTPUT_FILE="decrypted.txt" # Archivo donde se guardará el texto descifrado

#Crear el archivo luffy_xor.py con el contenido correcto
cat > luffy_xor.py << 'EOF'
def xor_cipher(text, key):
    # Convertir tanto el texto como la clave a bytes
    if type(text) == str:
        text_bytes = text.encode()
    else:
        text_bytes = text
    key_bytes = key.encode() # Convertir la clave a bytes

    # Cifrar con XOR
    cipher_text = bytes([text_bytes[i] ^ key_bytes[i % len(key_bytes)] for i in range(len(text_bytes))])

    return cipher_text
EOF

#Crear un script Python temporal para la extracción
cat > extract_script.py << 'EOF'
from PIL import Image
import piexif
import sys
from luffy_xor import xor_cipher

def extraer_texto_metadata(imagen_path):
```

```

# Abrir la imagen
img = Image.open(imagen_path)

# Obtener los metadatos EXIF
try:
    exif_data = img.info.get('exif')
    if not exif_data:
        return None

    exif_dict = piexif.load(exif_data)

    # Obtener el texto almacenado en 'Artist'
    texto = exif_dict['0th'].get(piexif.ImageIFD.Artist)
    if texto:
        return texto
    return None
except Exception as e:
    print(f"Error al extraer metadatos: {e}")
    return None

# Obtener argumentos
if len(sys.argv) != 4:
    print("Uso: python script.py <imagen_path> <carne> <output_file>")
    sys.exit(1)

imagen_path = sys.argv[1]
student_id = sys.argv[2]
output_file = sys.argv[3]

# Extraer y descifrar
texto_cifrado = extraer_texto_metadata(imagen_path)
if texto_cifrado:
    try:
        # Aplicar XOR y decodificar si es posible
        texto_descifrado = xor_cipher(texto_cifrado, student_id)
        try:
            # Intentar decodificar como UTF-8
            decoded_text = texto_descifrado.decode('utf-8')

```

```

# Guardar en archivo
with open(output_file, 'w') as f:
    f.write(decoded_text)

# Mostrar en pantalla
print(decoded_text)

except UnicodeDecodeError:
    # Si falla la decodificación UTF-8, guardar como bytes en archivo bina
    with open(output_file, 'wb') as f:
        f.write(texto_descifrado)

    # Mostrar en pantalla
    print(texto_descifrado)
    print(f"\nEl contenido binario descifrado se ha guardado en: {output_fil

except Exception as e:
    print(f"Error al descifrar: {e}")
else:
    print("No se encontraron metadatos EXIF o no hay campo Artist en la image
EOF

#Ejecutar el script en el mismo contenedor Docker
pip install pillow piexif

python3 extract_script.py "$IMAGE_PATH" "$STUDENT_ID" "$OUTPUT_FILE"

echo "Texto descifrado guardado en: $OUTPUT_FILE"

rm -f luffy_xor.py extract_script.py

```

Creamos el script utilizando el sexto comando documentado en  [Utils](#) para luego otorgarle permisos de ejecución utilizando el séptimo comando documentado.

Finalmente, ejecutamos el script utilizando el octavo comando documentado en  [Utils](#).


```
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks/poneglyph$ nano decryptImg.sh
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks/poneglyph$ chmod +x decryptImg.sh
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks/poneglyph$ ./decryptImg.sh "21542"
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pillow in /home/zoro/.local/lib/python3.10/site-packages (11.2.1)
Requirement already satisfied: piexif in /home/zoro/.local/lib/python3.10/site-packages (1.1.3)
he knows what the Void Century is and that Roger solved the mysteries of the Poneglyphs;
Texto descifrado guardado en: decrypted.txt
zoro@1241ab859171:~/ONEPIECE/Pirate_Island/Pirates_Tavern/Casa_de_Shanks/poneglyph$
```

Obteniendo así, el párrafo descriptado.

he knows what the Void Century is and that Roger solved the mysteries of the